计算机图形学实验报告

实验三：Pipeline and Shading（管线与着色）

姓名：刘博文　学号：09023225

2025 年 11 月 13 日

1　实验目的

本实验旨在深入理解图形学渲染管线中的着色与光照过程。通过编程实现光栅化阶段中的多种着色方法，掌握以下内容：

- 使用重心坐标实现颜色、法向量、纹理坐标等属性插值；
- 理解并实现 Blinn–Phong 光照模型；
- 在 Phong 模型基础上实现纹理映射（Texture Mapping）；
- 实现凹凸映射（Bump Mapping）与位移映射（Displacement Mapping）；
- 掌握 Vertex Shader 与 Fragment Shader 的运行机制。

2　实验环境

- 操作系统：Ubuntu 18.04 (课程虚拟机)
- 编译工具：CMake + GNU Make
- 编译命令：

```
mkdir build
cd build
cmake ..
make
```

- 运行命令示例：

```
./Rasterizer normal.png normal
./Rasterizer phong.png phong
./Rasterizer texture.png texture
./Rasterizer bump.png bump
./Rasterizer displacement.png displacement
```

3 实验内容与实现步骤

3.1 参数插值实现（rasterize_triangle）

在 `rasterizer.cpp` 中，对每个三角形内的像素计算重心坐标 (α, β, γ)，并对颜色、法向量、纹理坐标、视空间位置等属性进行插值：

$$P = \alpha P_0 + \beta P_1 + \gamma P_2$$

然后将结果传入 `fragment_shader_payload` 中，以供片元着色器使用。

3.2 透视投影矩阵（get_projection_matrix）

采用透视投影变换矩阵：

$$M_{proj} = \begin{bmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{-(f+n)}{f-n} & \frac{-2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

其中 n, f 分别为近平面和远平面距离。

图 1: normal 示意图

3.3 Blinn–Phong 光照模型

Phong 模型的反射计算公式为:

$$I = I_a k_a + \sum_i I_i \left[k_d \max(0, n \cdot l_i) + k_s \max(0, n \cdot h_i)^p \right]$$

其中:

- I_a: 环境光强度;
- k_a, k_d, k_s: 环境、漫反射、镜面反射系数;
- l_i: 光源方向;
- $h_i = \frac{v+l_i}{||v+l_i||}$: 半程向量;
- p: 高光系数。

实现核心代码如下:

```
for (auto& light : lights) {
    Vector3f l = (light.position - point).normalized();
    Vector3f v = (eye_pos - point).normalized();
    Vector3f h = (l + v).normalized();
    float r2 = (light.position - point).squaredNorm();
```

```
    Vector3f ambient = ka.cwiseProduct(amb_light_intensity);
    Vector3f diffuse = kd.cwiseProduct(light.intensity / r2) * max
        (0.0f, normal.dot(l));
    Vector3f specular = ks.cwiseProduct(light.intensity / r2) * pow(
        max(0.0f, normal.dot(h)), p);

    result_color += ambient + diffuse + specular;
}
```

图 2: Blinn–Phong 光照结果示意图

3.4 纹理映射（Texture Mapping）

在 Phong 模型基础上，将纹理颜色作为漫反射系数 k_d:

```
if (payload.texture) {
    Vector3f tex_color = payload.texture->getColor(u, v);
    kd = tex_color / 255.f;
}
```

图 3: Texture Mapping 效果图

3.5 凹凸映射（Bump Mapping）

不改变几何位置，仅扰动法线方向来模拟凹凸效果。

$$\mathbf{ln} = (-dU, -dV, 1)$$

$$n' = \text{normalize}(TBN \cdot \mathbf{ln})$$

其中：

- $TBN = [t, b, n]$ 为切线空间变换矩阵；
- $dU = k_h k_n [h(u + 1/w, v) - h(u, v)]$；
- $dV = k_h k_n [h(u, v + 1/h) - h(u, v)]$。

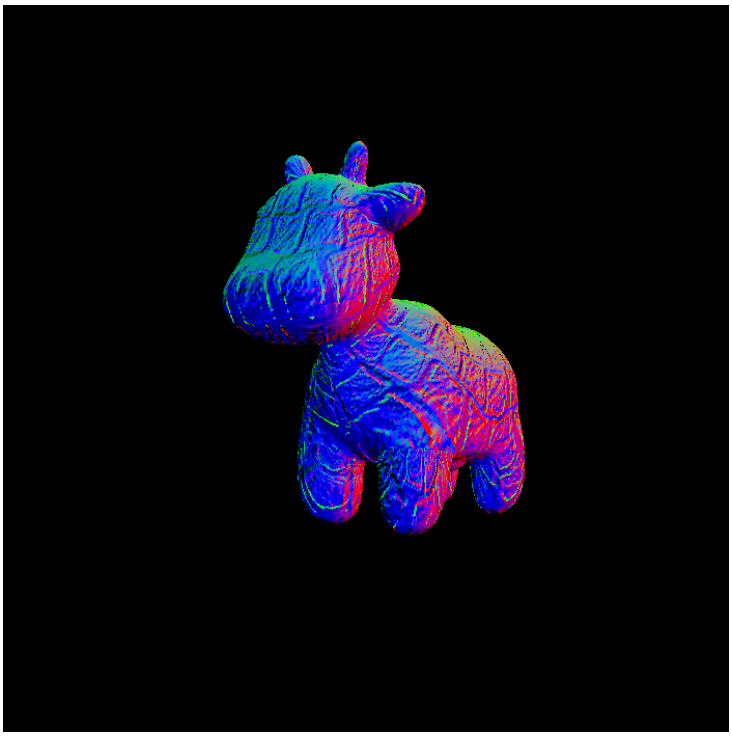

图 4: Bump Mapping 法线扰动效果

3.6 位移映射（Displacement Mapping）

位移映射在凹凸映射的基础上进一步修改顶点位置：

$$p' = p + k_n n h(u, v)$$

然后计算扰动法线 n'，并使用新的 p' 进行光照计算。该方法产生了真实几何凹凸效果。

图 5: Displacement Mapping 真实几何起伏效果

4 实验结果对比

表 1: 不同着色方式效果描述

着色方式	效果描述
Normal Shader	按法向量上色，展示表面方向
Phong Shader	具有高光的光照效果
Texture Shader	显示真实纹理材质
Bump Mapping	光照变化模拟表面凹凸
Displacement Mapping	表面几何真实起伏

5 实验总结

通过本次实验，我深入理解了现代图形学中从顶点到像素的完整渲染管线。主要收获如下：

1. 掌握了重心坐标插值及其在光栅化中的应用；
2. 理解了 Blinn–Phong 模型的物理意义及参数作用；
3. 掌握了纹理映射、凹凸映射、位移映射的原理与实现区别；

4. 实际体验了 Fragment Shader 中的光照计算流程；
5. 理解了 GPU 渲染的流水线结构：Vertex Shader → Fragment Shader → Framebuffer。

本次实验提升了我对光照模型、着色算法及图形渲染过程的整体理解。